Maintenance and Recovery Handbook

Safe IIS deployment, upgrades, validation and recovery

Portfolio context

Developed during my IT internship at Murphy Ben International (MBI) as an internship/internal experimental project for collecting and analyzing broadcast signal reports. I was responsible for the design and implementation of the application, working under the guidance of a senior member of the IT team.

This repository is a portfolio representation of the project. Sensitive configuration, internal information, production data, credentials, and company-specific deployment details have been excluded.

1. Deployment baseline

```
https://signal-logger.example.com/ -> Public
https://signal-logger.example.com/field/ -> Field
https://signal-logger.example.com/admin/ -> Admin
/api/* -> IIS ARR -> 127.0.0.1:3000
```

IIS responsibilities

- HTTPS/TLS termination on port 443.
- Static frontend delivery.
- URL Rewrite and ARR reverse proxy for /api/*.
- HTTP-to-HTTPS redirection.
- HSTS only after HTTPS has been verified.

2. Pre-deployment backup

```
server/config.json
server/incidents.json
server/public_incidents.json
server/audit.json
server/id_sequences.json
web.config
```

Runtime JSON is operational data, not deployment content. Never replace it with example templates during an upgrade.

3. Safe upgrade sequence

Step	Action
1	Back up the active deployment and runtime data
2	Deploy the smallest required file set
3	Deploy server changes before clients that depend on them
4	Restart Node.js only when backend code changed
5	Verify /api/health
6	Hard-refresh affected frontend pages
7	Confirm new and historical records remain available

4. Validation checklist

- Public, Field and Admin routes load over HTTPS.
- HTTP redirects to HTTPS.
- The certificate hostname and validity are correct.
- Node.js remains bound to 127.0.0.1.
- PUB and INC submissions receive server-generated IDs.
- Admin sees both incident sources.
- CSV, print and KML exports work.
- Runtime files retain correct permissions and content.

5. Recovery procedure

- Stop the affected service if writes could continue.
- Restore only the code or configuration files known to have changed.
- Restore runtime data from the latest verified backup only when necessary.
- Restart Node.js only if backend files were restored.
- Check /api/health, submit test reports and confirm historical data.
- Review IIS and application logs before closing the incident.

6. Common faults

Symptom	Check
Frontend loads but API fails	ARR proxy, rewrite rule, Node.js service and localhost:3000
GPS unavailable	HTTPS, browser permission and device location services
Old frontend remains visible	Hard refresh, site data and Field service worker
Reports do not persist	JSON validity, file permissions and available storage
HTTPS warning	Binding, hostname, certificate chain and system time